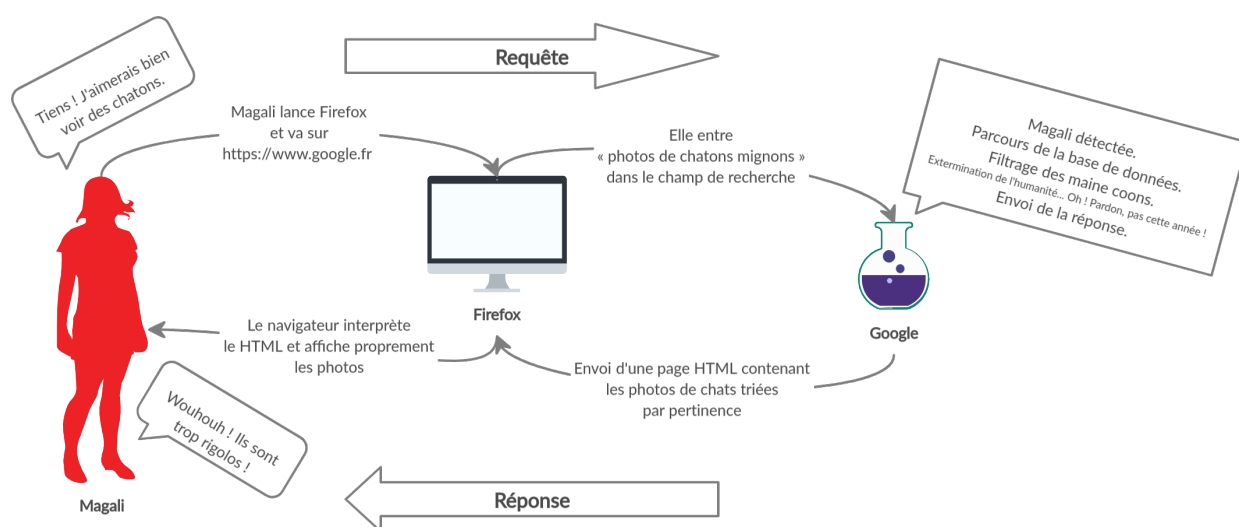


Le Backend

Frontend

Souvenez-vous de la recherche de Magali.



Magali envoie une requête HTTP à un serveur : « photos de chatons mignons ». Elle envoie cette requête à un serveur à travers l'Internet. Le serveur réceptionne la requête, la traite, puis génère une page HTML. Il envoie la page HTML au navigateur de Magali, ainsi que tous les fichiers annexes (*assets*) demandés par ledit HTML : images, polices, CSS, javascript, etc. S'il y a du CSS, le navigateur va l'interpréter pour donner forme au HTML. S'il y a du javascript, le navigateur va l'exécuter pour créer magie et interaction avec Magali.

HTML, CSS, javascript, tous ces fichiers ont été préparés en amont par les développeurs. Le but de ces fichiers est de constituer une réponse à une requête, et d'être compris et affichés par un navigateur. Tout ce travail d'intégration de maquettes et de développement JS relève de ce que l'on appelle le *frontend*. On développe ce qui va être vu par l'utilisateur, ce qui va interagir directement avec lui à travers son navigateur.

À ce point, nous comprenons désormais ce qu'est le *frontend*. Mais il reste une part d'obscurité dans notre compréhension du web : le côté serveur. Comment le HTML est-il généré ? Comment, dans notre schéma, le serveur fait-il pour reconnaître Magali ? Pour parcourir une base de données ? Pour filtrer les maine coons ? Toute cette logique est plus

ou moins transparente pour l'utilisateur, ce sont les coulisses du web, autrement dit le *backend*.

Backend

Le *backend* est la partie de notre travail de développeurs qui consiste à mettre en place cette logique de génération de HTML. Les intégrateurs ont préparé des *templates*, des pages HTML avec des trous ; les développeurs *backend* vont devoir renvoyer les bonnes pages en fonction de ce que l'utilisateur demande, et remplir ces trous dans les *templates* avec des données adéquates.

L'utilisatrice a demandé des chatons, mais il faut qu'ils soient mignons. Et c'est Magali, elle n'aime pas les maine coons. Son navigateur est en français, il va falloir privilégier les résultats français...

Toute une logique !

Toute cette logique, toute cette algorithmie, il va falloir l'exprimer de sorte à ce que notre serveur puisse l'exécuter. Ici intervient donc un langage de programmation orienté *backend* : PHP.

Et si le javascript est plus ou moins le seul langage de programmation compris par les navigateurs et utilisé en *frontend*, le nombre de langages *backend* est quant à lui immense. Java, Elixir, Go, Ruby... pour en citer quelques-uns. Et celui qui s'est historiquement imposé comme le langage le plus utilisé dans les *backends* du web : le PHP.

C'est un langage en constante évolution, avec un écosystème et une communauté gigantesques. Si PHP 5.6 a pu montrer des signes de fatigue sur le marché il y a quelques années face à des concurrents comme Node, sa version 7 sortie fin 2015 lui a fait reprendre du poil de la bête et il est aujourd'hui au sommet de sa forme.

Voyons cela !

Variables

Comme en javascript, on peut déclarer des variables pour stocker des valeurs en mémoire vive. Il n'y a qu'une seule façon de faire :

```
$x = 4;  
$pi = 3.14;  
$msg = "Salut";
```

À la différence du javascript, le PHP ne nécessite pas de mot-clé comme `let` ou `const`, mais le signe `$` devant le nom d'une variable.

À savoir que les variables en PHP sont mutables. PHP ne nous permet malheureusement pas de déclarer des variables immutables comme les `const` en javascript.

```
$msg = "Salut";  
$msg = "Yo";  
echo($msg);
```

On peut réassigner à volonté la valeur d'une variable, donc ce code affichera `Yo`.

Notez qu'il existe bien des constantes en PHP. Nous y reviendrons quand nous aborderons la programmation orientée objet.

Quelques mots sur les références

Vous pourrez parfois être amenés à voir la forme suivante :

```
$msg1 = "Salut";  
$msg2 = &$msg1;
```

Notez l'usage de l'esperluette `&`. Il s'agit de ce que l'on appelle une référence. Ici, la variable `$msg2` ne devient pas une copie `$msg1` comme vous pourriez vous y attendre : elle crée un alias qui pointe vers la même valeur `"Salut"`. Nous ne nous retrouvons donc pas avec deux variables contenant chacune une copie de `"Salut"`, mais bien une valeur unique `"Salut"` pouvant être appelée par deux noms différents.

```
function increment(&$x) {  
    ++$x;  
}
```

```
$number = 1;  
increment($number);  
echo($number);
```

Ceci est un passage par référence. En ajoutant `&` devant un argument lors de la définition d'une fonction, vous faites en sorte que cet argument soit passé en tant que référence et non en tant que copie à l'appel de la fonction. En somme, si vous modifiez la valeur de cet argument à l'intérieur de la fonction, elle sera également modifiée à l'extérieur. Ici, le code affichera `2`.

Nous vous montrons les références parce qu'elles existent. Vous les verrez parfois dans le code d'autres développeurs. Sachez toutefois que nous ne les utiliserons plus dans ce cours. En effet, il convient d'avoir une excellente compréhension du fonctionnement profond de PHP pour pouvoir utiliser intelligemment les références. PHP est déjà plus malin que l'on peut l'imaginer dans sa gestion de la mémoire, notamment grâce à la notion de *copy-on-write*. Un développeur inexpérimenté utilisant les références avec les meilleures intentions du monde amènera beaucoup plus de problèmes que de solutions. Les références sont le plus souvent sources de bugs et de complexité inutile du code. Sachez que, dans les cas d'usage que vous rencontrerez au début de votre carrière, les références seront toujours une mauvaise idée, révélatrices d'un mauvais design.

Types

On retrouve en PHP les types du javascript, à quelques différences près.

Chaînes de caractères

Il existe deux manières de déclarer une chaîne de caractères :

```
$txt1 = "Salut";  
$txt2 = 'Salut';
```

Guillemets simples `' '` ou doubles `""`. Quelle différence ? Il faut parler de concaténation pour comprendre.

```
$place = 'la planète';  
$msg = 'Salut ' . $place;  
echo($msg);
```

Ce morceau de code affichera **Salut la planète**. Avec des chaînes à guillemets simples `' '`, on concatène deux chaînes grâce au signe `.`, contrairement au javascript qui utilise le signe `+`. Mais des chaînes à guillemets doubles `""` permettent d'intégrer **les variables** directement à la chaîne :

```
$place = 'la planète';  
$msg = "Salut $place";  
echo($msg);
```

Sympa ! On peut aussi l'écrire avec des accolades.

```
$place = 'la planète';  
$msg = "Salut {$place}";  
echo($msg);
```

Les accolades sont ici peu utiles et servent uniquement à la lisibilité, mais elles sont nécessaires dans certains cas, comme un appel de méthode sur un objet par exemple :

```
$user = new User();  
$msg = "Salut {$user->getFirstName()}";  
echo($msg);
```

Dans tous les cas, au bout du compte, une *string* reste une *string* et le type de la variable sera exactement le même qu'on l'ait déclarée avec `""` ou avec `' '`

Si les guillemets doubles `""` sont si utiles, pourquoi ne pas les utiliser tout le temps ?

Parce que quand PHP voit des `""`, il analyse systématiquement la chaîne pour y trouver des variables, même s'il n'y en a pas. Utiliser des `""` sur une chaîne aussi simple que **Salut** conduit donc à une analyse de texte inutile et une légère surconsommation des **ressources** du processeur. En bref, les guillemets simples `' '` sont plus performants et donc préférables quand on n'a pas besoin de concaténer des variables. Meilleures performances, moins d'électricité, moins de CO2.

Nombres

Contrairement au javascript qui emploie un type unique **Number**, le PHP distingue le type **integer** (ou **int**) du type **float**, autrement dit les nombres entiers des nombres décimaux.

Quand vous comptez les personnes présentes dans une pièce, on vous entendra ainsi : « Un, deux, trois, quatre, [...] ». Une personne vaut toujours **1**. Vous pouvez les additionner, les multiplier, etc., vous n'arriverez jamais à une fraction. Nombres entiers, ou **integer** en PHP.

Quand vous prenez les mesures d'une fenêtre pour acheter des rideaux, vous trouverez potentiellement des valeurs telles que **40,25** centimètres de hauteur et **80,55** centimètres de largeur. Si par hasard vous aviez besoin de calculer l'aire de cette fenêtre (on a le droit de s'ennuyer), vous procéderiez au calcul **40.25 × 80.55** pour obtenir le résultat de **3242,1375** centimètres, que vous pourriez même arrondir à **3242,14**, convertir en mètres, etc. Nombres décimaux, ou **float** en PHP.

En dehors de cette légère distinction au niveau du type, la syntaxe ne diffère pas du javascript :

```
$x = 12;  
$y = 45.263;
```

Booléens

Les booléens **true** et **false** existent toujours. C'est le type **bool**, ou **boolean**.

```
$isUserAnonymous = false;
```

```
if (!$isUserAnonymous) {  
    echo('Salut Jacques !');  
}
```

Vous les verrez parfois orthographiés en majuscules : **TRUE** et **FALSE**. C'est une ancienne habitude du C que le PHP autorise, mais on aura tendance à préférer les formes minuscules dans du code moderne.

Null

Le type **null**, qui ne peut être que de la valeur **null**, est également présent.

```
$user = null;
```

```
if ($user === null) {  
    echo('Pas d'utilisateur !');  
}
```

Comme pour les booléens, une forme majuscule est possible : **NULL**. Et comme pour les booléens, c'est une forme un peu âgée qu'on évitera à moins de vraiment aimer la déco kitsch.

Tableaux

Un tableau PHP :

```
$numbers = [4, 6, 8];
```

Vous verrez parfois l'ancienne syntaxe :

```
$numbers = array(4, 6, 8);
```

Cette façon de déclarer un tableau fonctionne toujours et ne présente aucune différence en termes de performances. De nos jours on va toutefois avoir tendance à favoriser les crochets `[]`. Ils sont plus courts et sont utilisés dans beaucoup d'autres langages pour représenter les tableaux, y compris le javascript, et sont donc visuellement plus intuitifs. Dans tous les cas, comme toujours, privilégiez de suivre le style et les conseils de vos formateurs.

```
$texts = ['Salut', 'Yo', 'Hey'];  
echo($texts[0]);
```

Ce morceau de code affichera **Salut**. La variable `$texts` contient un tableau à index numériques. Comme ceux du javascript, les clés de chaque valeur sont donc numérotées dans l'ordre de 0 à l'infini.

Notez qu'en PHP, un tableau n'utilise pas forcément des index numériques.

```
$user = [  
    'firstName' => 'Magali',  
    'age' => 22,  
    'city' => 'Rennes'  
];
```

Nous avons ici ce que l'on appellera un tableau associatif. Contrairement au tableau à index numériques, le tableau associatif n'utilise pas des clés numériques mais des *strings* arbitrairement fixées par le développeur.

Et comment accède-t-on à une valeur dans un tableau associatif ?

```
$values = [  
    'vegetable' => 'Échalote',  
    'fruit' => 'Fraise'  
];
```

```
$x = $values['vegetable'];
```

```
echo($x);
```

Ce code affichera **Échalote**.

Bien que les clés soient différentes, les tableaux à index numériques et les tableaux associatifs relèvent du même type pour le PHP : c'est toujours un **array**.

Objets

Les objets (type **object**) existent évidemment en PHP. Bien qu'officiellement le PHP soit un langage multi-paradigmes, officieusement il s'est imposé à travers les années comme un langage orienté objet. Presque tout son écosystème est développé autour de la POO.

L'objet en PHP est un sujet trop complexe pour être abordé rapidement dans une sous-section. Nous y reviendrons vite en détails.

Autres

Il existe quelques autres types en PHP, tels que **iterable** et **callable**. Vous n'avez pas forcément besoin de les connaître tout de suite, la curiosité et le code vous y mèneront bien assez vite.

Conversions

Il existe plusieurs façons de pratiquer des conversions de types en PHP. Vous rencontrerez parfois, par exemple, la fonction **intval**. La syntaxe que nous allons vous montrer ici est cependant la plus intuitive et la plus cohérente, car c'est la même façon de faire pour tous les types :

```
$a = '1'; // '1'
$b = (int) '1'; // 1
$c = (float) '1'; // 1.0
$d = (bool) '1'; // true
```

Très simple !

Jetez donc un œil à la **documentation** pour avoir un aperçu des différents *casts* possibles. On pensera notamment à **(string)**, **(array)**, etc.

On retrouve cette syntaxe de conversion dans de nombreux langages comme le C, le java ou encore le C#. Elle est donc visuellement intuitive pour la plupart des développeurs.

Comparaisons et conditions

Les conditions en PHP sont identiques au javascript, et on y retrouve les mêmes opérateurs de comparaison dans les expressions booléennes : `<`, `<=`, `>`, `>=`, `!=`, `!==`, `==`, `===`.

```
$msg = 'Salut';  
if ($msg === 'Salut') {  
    echo($msg);  
}
```

La recommandation concernant la comparaison d'égalité reste la même qu'en javascript : vous n'aurez jamais réellement besoin de `==` et `!=`, ces opérateurs sont sources de bugs ; préférez toujours `===` et `!==`.

Le PHP a aussi deux opérateurs supplémentaires :

- La différence `<>`. Ce signe est très peu utilisé et a le même comportement que `!=`. À éviter, donc.
- Le *spaceship operator* `<=>`, qui sert aussi à comparer des nombres. Contrairement aux autres opérateurs, `<=>` ne renvoie pas un booléen mais un nombre qui peut être `-1`, `0` ou `1` : `-1` si le nombre de gauche est inférieur, `1` si le nombre de gauche est supérieur, `0` si les deux nombres sont égaux. En cela, ce n'est pas vraiment une expression booléenne, on ne l'utilise pas dans les conditions. Son usage est assez anecdotique, on l'emploie par exemple dans la fonction `usort` pour trier un tableau.

Switch

Un *switch* en PHP :

```
$x = 0;  
switch ($x) {  
    case 0:  
        echo('Salut');  
        break;  
    case 1:  
        echo('Yo');  
        break;  
    default:  
        echo('Hey');  
}
```

Ce code affichera `Salut`.

Les Boucles

Meanwhile in PHP

La bonne nouvelle avec les structures de contrôle telles que les conditions et les boucles, c'est que la logique est sensiblement la même d'un langage impératif à un autre. Seule la syntaxe est souvent amenée à changer. Qu'on dise « chocolatine » ou « pain au chocolat », le goût sera vraisemblablement le même.

Ce que vous avez appris en javascript au sujet des boucles est donc très valable en PHP, à quelques différences près. Faisons un petit tour d'horizon.

Boucle *while*

```
$i = 1;
```

```
while ($i < 4) {  
    echo $i;  
    ++$i;  
}
```

Rien de nouveau. La boucle *while* continue de tourner tant que sa condition est évaluée à *true*. Exactement le même fonctionnement que la boucle *while* du javascript.

Boucle *for*

```
$values = ['A', 'B', 'C', 'D'];
```

```
for ($i = 0, $max = count($values); $i < $max; ++$i) {  
    echo $values[$i];  
}
```

Notre tableau à index numériques est parcouru de la même manière qu'on parcourt un *Array* en javascript. On déclare un itérateur, et on l'incrémente à chaque tour de boucle en s'en servant comme clé numérique, jusqu'à atteindre la taille totale du tableau.

Conseil d'optimisation

Notez la déclaration d'une variable *\$max* au premier tour de boucle. L'appel de la fonction *count* est en effet coûteux en performances, aussi on ne le fait qu'une seule fois et on stocke le résultat dans une variable plutôt que demander au processeur de recalculer cette valeur à chaque tour de boucle.

Boucle *foreach*

La boucle `foreach` en PHP est plus ou moins l'équivalent de la boucle `for ... of` en javascript, pour dire les choses simplement, mais elle a tout de même beaucoup de particularités.

Elle permet notamment de parcourir un tableau associatif. En effet, si une boucle `for` avec un itérateur est suffisante pour un tableau à index numériques, par nature elle ne peut pas boucler à travers des index associatifs.

```
$values = [  
    'msg' => 'Salut',  
    'name' => 'Jacques',  
    'age' => 45,  
    'city' => 'Rennes',  
];  
  
foreach ($values as $key => $value) {  
    echo("Clé : $key.");  
    echo('<br/>');  
    echo("Valeur : $value.");  
}
```

Pas mal. À chaque itération, la boucle `foreach` donne accès à la valeur courante du tableau, mais aussi à sa clé. La syntaxe `$values as $key => $value` pourrait se traduire ainsi : « Parcoure le tableau `$values`. À chaque itération, la clé courante s'appellera `$key`, et la valeur courante s'appellera `$value` ». `$values` est donc le tableau à parcourir, là où `$key` et `$value` sont ici des variables arbitrairement choisies par le développeur pour y assigner respectivement la clé et la valeur de l'itération en cours.

Notez que si nous n'avons pas besoin de la clé, nous pouvons l'omettre.

```
$letters = ['a', 'b', 'c', 'd'];  
  
foreach ($letters as $letter) {  
    echo($letter);  
}
```

Nous n'avons pas besoin de la clé, nous pouvons omettre son assignation pour n'écrire que `$letters as $letter`.

Hé ! Mais c'était pas un tableau associatif, ça.

Bien sûr. Rien ne nous empêche d'utiliser la boucle `foreach` pour parcourir un tableau à index numériques. On l'a dit, en PHP un tableau est un `array`, qu'il soit associatif ou non ne change pas son type. Pour être plus précis, il relève même du pseudo-type `iterable`, et toute valeur du type `iterable` peut être traversée par une boucle `foreach`. En PHP, les tableaux sont `iterable`, mais

c'est aussi le cas des objets implémentant l'interface `\Traversable`. Nous en avons déjà trop dit, tout cela devient un peu velu.

Pour l'heure, retenez simplement que `foreach` permet de parcourir tous les tableaux, qu'ils soient associatifs ou non.

Boucle *do ... while*

Nous ne détaillerons pas ici la boucle `do-while`. C'est une boucle `while` dont la particularité est d'exécuter systématiquement une première fois son bloc d'instructions avant même de tester sa condition. Le bloc d'instructions sera donc toujours au moins exécuté une fois, même si la condition est fausse dès le départ.

Quand utiliser quoi ?

La boucle `while` s'utilise rarement pour parcourir un tableau. On l'emploie plutôt pour boucler autour d'une vraie condition booléenne. Souvenez-vous par exemple de l'exercice en javascript où l'on continuait à boucler tant que l'utilisateur n'avait pas entré la bonne valeur. La boucle `while`, c'est littéralement *tant que*. En vérité vous la verrez assez peu souvent dans le contexte du PHP : comme le `do-while` elle dépend philosophiquement d'un changement d'état parallèle à son exécution, ce qui arrive très anecdotiquement dans un langage purement synchrone comme le PHP, où il faudra généralement qu'elle provoque elle-même ce changement d'état.

Notez qu'en soi rien n'empêche d'utiliser `while` avec un itérateur, mais à ce moment autant utiliser un `for` dont la syntaxe *inline* est plus adaptée à ce besoin.

La boucle `for` s'utilise essentiellement avec un itérateur. On a vu que l'itérateur pouvait servir à parcourir un tableau, mais ce n'est pas son seul usage. Le nombre d'itérations peut être arbitrairement fixé par le développeur pour exécuter une certaine action un certain nombre de fois. Vous serez par exemple amenés à développer des fonctions dont le rôle pourrait être de créer 1000 faux comptes utilisateurs en base de données à des fins de tests.

La boucle `foreach` est exclusivement utilisée pour parcourir un *iterable*, ce qui inclut les tableaux. À ce titre elle est aussi plus lisible et moins sujette à bugs, même pour parcourir des tableaux à index numériques. Et elle est très performante : pour des raisons très techniques qu'on ne détaillera pas ici, en lecture seule, tant qu'on ne modifie pas le tableau, le `foreach` peut même être plus rapide qu'un `for`.

En résumé :

- `while` pour exécuter un bloc d'instructions tant qu'une condition factuelle est remplie, sans itérateur ;
- `for` pour exécuter un bloc d'instructions un certain nombre de fois, ou pour parcourir un tableau à index numérique ;

- `foreach` pour parcourir tous les tableaux.

Dans tous les cas, encore et toujours : suivez les préférences et conseils de vos formateurs.

Fonctions...

De base, une fonction PHP peut être déclarée de la même façon qu'en javascript.

```
function multiply($x, $y) {  
    return $x * $y;  
}
```

...mais en PHP, il est de bonne pratique de typer les arguments et le return. Cela s'appelle le type hinting.

Le *type hinting*

En PHP, les `types` sont très importants. Le PHP permet de forcer les arguments d'une fonction à être de tel ou tel type, et il peut même forcer la fonction à ne renvoyer que tel ou tel type.

```
function addExclamationToMessage(string $message): string {  
    return $message . '!';  
}
```

Une fonction très (très) utile qui ajoute un point d'exclamation à une chaîne de caractères. L'argument `$message` sera obligatoirement de type `string`, et la valeur de retour sera elle aussi obligatoirement de type `string`.

On traite un message, et on renvoie ce message modifié. En somme, on traite une `string`, et on renvoie une `string`. Là où en javascript il fallait faire confiance au développeur pour ne pas se tromper et renvoyer par exemple un nombre, le PHP permet de forcer le programme à ne pas accepter autre chose que le type explicitement requis.

Ce comportement s'appelle le *type hinting*. Par exemple, le code suivant causera immédiatement un bug :

```
function addExclamationToMessage(string $message): string {  
    return [$message, '!'];  
}
```

PHP attend une `string` en retour, il ne va pas être content quand il verra qu'on tente de renvoyer un `array`. Il lèvera une erreur sans attendre. Cela permet de chasser un bug très en amont, en ne laissant même pas une valeur erronée quitter sa fonction. Si on laissait cette fonction `addExclamationToMessage` renvoyer un tableau, cela pourrait au bout du

compte induire un bug très sournois et difficile à traquer bien plus loin dans l'exécution du programme.

En typant les valeurs d'entrée et de sortie des fonctions, on tue de très nombreux bugs potentiels dans l'œuf.

Si le *type hinting* n'est techniquement pas obligatoire, dans le sens où PHP ne l'impose pas, c'est bien plus qu'une simple bonne pratique : c'est devenu indispensable dans le monde professionnel.

Types nullable

Depuis sa version 7.2 (2017), PHP propose une syntaxe très pratique pour dire qu'une valeur peut être soit d'un certain type, soit `null`. Le type `null` représentant l'absence de valeur, il se marie bien avec les autres types pour dire par exemple « cette valeur est soit un nombre soit rien du tout ».

Pour spécifier un type nullable, on ajoute simplement un point d'interrogation devant le nom du type : `?string`, `?int`, etc. Exemple :

```
function getNameFromUser(array $user): ?string {  
    if (isset($user['name']) && is_string($user['name'])) {  
        return $user['name'];  
    }  
  
    return null;  
}
```

Si le tableau `$user` contient bien un index `name`, et que la valeur trouvée à cet index est bien une `string`, alors on la retourne. La fonction retourne alors une `string`.

Si, au contraire, le tableau `$user` ne contient pas d'index `name`, ou que cet index `name` existe mais présente autre chose qu'une `string`, alors la fonction retourne `null`.

Nous avons donc une fonction qui retourne soit une `string` soit `null`. Son retour est par conséquent typé à `?string`.

Notez que si cela fonctionne avec les valeurs de retour, c'est aussi valable pour les valeurs d'entrée.

Valeurs par défaut

Comme en javascript, on peut définir des valeurs par défaut pour les arguments.

```
function appendPunctuation(string $str, string $punctuation = '.'): string {  
    return $str . $punctuation;  
}  
  
echo appendPunctuation('Salut');
```

En omettant ici le deuxième argument lors de l'appel de la fonction, on le laisse utiliser sa valeur par défaut `'.'`. Ce code affiche donc `Salut..`

Les Conventions

Les conventions c'est bien

Nous disions qu'en javascript les développeurs ont souvent leurs préférences personnelles sur beaucoup de sujets. Point-virgule ou pas point-virgule, `let` ou `const`, etc.

Nous disions aussi que le PHP était un monde beaucoup plus normé. On y trouve en effet une convention mondialement acceptée et appliquée qui est le `PSR-12` (anciennement `PSR-2`).

Nous vous invitons à jeter un œil à la spécification `PSR-12`. Il s'agit d'un ensemble de règles de style à essayer de respecter au maximum dans un code professionnel. Indentation à 4 espaces, espace entre le `if` et la parenthèse, variables et fonctions en *camelCase*, etc. Autant de règles provenant du `PSR-12` et appliquées par la très large majorité des développeurs PHP à travers le monde.

Mais pourquoi respecter des règles ?

Le fait de respecter des conventions syntaxiques uniformes permet de rendre nos codes plus intuitifs. Quand vous lisez le code d'un développeur qui respecte les mêmes conventions que vous, vous comprenez très vite sa logique, c'est un style que vous avez l'habitude de voir. Vous pouvez ainsi facilement travailler avec des développeuses et développeurs de tous les pays et de toutes les entreprises sans aucun problème, et c'est d'ailleurs aussi pour cette raison que l'on code toujours en anglais.

Un code qui ne respecte aucune convention peut même être rédhibitoire pour un potentiel futur employeur qui regarderait votre travail. Si quelques libertés stylistiques seront toujours défendables, et vous en prendrez évidemment parfois, un code totalement *freestyle* n'aura pour effet que de refléter un certain amateurisme aux yeux d'un professionnel.

Moi je moi je

Et moi si je veux avoir mon propre style ?

Vous aurez votre propre style. Le PSR-12 ne couvre pas absolument tous les cas de figure imaginables. Ce sont des conventions basiques, aussi simples que « fourchette à gauche et couteau à droite ».

Vous ne saluez pas les gens de la main gauche parce que vous suivez les conventions de votre monde. Savoir si vous mettez l'accolade à la ligne ou non en déclarant une fonction est secondaire. Ce n'est pas important, ce n'est pas représentatif de votre personnalité. Sur des questions aussi triviales vous respecterez autant que possible le PSR-12. Votre style personnel ne se situera dans de sombres histoires d'accolades et de parenthèses, mais dans votre approche des problèmes, dans votre logique, dans votre algorithmie. Vos collègues et vous-mêmes reconnaîtrez votre code à son architecture globale, à l'élégance de vos solutions, à votre façon de résoudre les problématiques que vous rencontrerez. Pas à vos retours à la ligne ou votre indentation.

Vous n'appliquerez probablement pas 100 % du PSR-12. De toute façon vous ne le connaîtrez pas par cœur, et personne ne vous demandera cela. On utilise même parfois des outils pour automatiquement formater notre code au PSR-12. Mais ayez l'humilité de vous dire que cette convention a été écrite par des personnes très compétentes qui avaient derrière chaque décision de très bons arguments.

Priorité au bon sens

Tout ceci étant dit, sachez tout de même qu'il faut savoir faire preuve de bon sens et de flexibilité. Il vous arrivera peut-être de travailler dans des équipes qui ne respectent pas le PSR-12, et ont au contraire des conventions internes qui leur sont propres. À ce moment, bien entendu, vous respecterez en priorité les consignes de vos collègues et le style des projets dans lesquels vous travaillerez.

echo

Après avoir parcouru quelques différences syntaxiques entre le javascript et le PHP, après avoir eu quelques mots autour du PSR-12, nous allons bientôt pouvoir commencer la pratique.

Avant les exercices, il va falloir au préalable comprendre comment fonctionne un fichier PHP.

À l'origine, le PHP est créé pour dynamiser le HTML. Au lieu de présenter des pages web statiques, le PHP permet de calculer à la volée certains morceaux de HTML. Prenons l'exemple suivant :

```
<!DOCTYPE html>
<html lang="fr-FR">
  <head>
    <meta charset="UTF-8">
    <title>Hey</title>
  </head>
  <body>
    <h1>Nous sommes le 30 07 2022</h1>
  </body>
</html>
```

Cette page affiche en titre principal la date. Mais à moins que les lois de la physique aient beaucoup changé depuis la première rédaction de ce cours, il y a de fortes probabilités pour que l'on ne soit plus le 30 07 2022.

```
<?php
```

```
function getCurrentDate(): string {
    return date('d m Y');
}
```

```
?>
```

```
<!DOCTYPE html>
<html lang="fr-FR">
  <head>
    <meta charset="UTF-8">
    <title>Hey</title>
  </head>
  <body>
    <h1>Nous sommes le <?php echo getCurrentDate(); ?></h1>
  </body>
</html>
```

Voilà, nous avons commencé à parsemer notre HTML de bouts de PHP.

En premier lieu nous déclarons notre fonction `getCurrentDate()`, qui ne fait pas grand-chose d'autre qu'utiliser la fonction PHP `date`.

Puis, dans notre fameux titre principal, nous utilisons cette fois la fonction `echo` pour afficher le retour de l'appel de `getCurrentDate`.

La fonction `echo` est donc primordiale en PHP, son rôle est d'afficher une chaîne de caractères dans du HTML. Même dans un programme qui contiendrait des dizaines de milliers de lignes de code, `echo` des résultats dans le HTML est le but final du PHP.

Afficher une chaîne de caractères dans du HTML, c'est tout ? C'est toute la base du web. Qu'est-ce qu'une page HTML sinon un grand fichier de texte ? Il s'agit dans tous les cas de construire du texte. Le PHP nous permet de dynamiser ce texte, de ne pas nous contenter d'un HTML statiquement fixé.

`echo` est l'une des rares fonctions où les parenthèses ne sont pas obligatoires. `echo('Salut')` fonctionne aussi bien que `echo 'Salut'`. Et on a d'ailleurs tendance à privilégier cette seconde syntaxe.

<?php ?> et syntaxe courte

Intégré à du HTML, le PHP est enclavé entre sa balise ouvrante `<?php`` et sa balise fermante ``?>`.

Il existe une exception à cela. Reprenons notre exemple :

```
<?php
```

```
function getCurrentDate(): string {  
    return date('d m Y');  
}
```

```
?>
```

```
<!DOCTYPE html>
```

```
<html lang="fr-FR">
```

```
    <head>
```

```
        <meta charset="UTF-8">
```

```
        <title>Hey</title>
```

```
    </head>
```

```
    <body>
```

```
        <h1>Nous sommes le <?= getCurrentDate() ?></h1>
```

```
    </body>
```

```
</html>
```

La syntaxe `<?php echo $value; ?>` peut être remplacée par `<?= $value ?>`. Sachant qu'un document HTML peut comporter beaucoup de `echo`, une syntaxe courte et lisible devient alors très pratique. Nous vous recommandons de l'utiliser.

Notez que dans le cas d'un fichier comportant exclusivement du PHP et aucun HTML, il n'est pas obligatoire – ni recommandé – de fermer la balise `<?php`. Mais ce n'est applicable que dans le cas où l'on sépare le PHP du HTML, nous reverrons cela dans les détails très vite.

Retenez pour l'instant que dans du HTML, les bouts de PHP sont toujours enclavés entre `<?php`` et ``?>`, sauf dans le cas de la syntaxe courte du `echo` : `<?=`` et ``?>`

Résumé

Vous devriez commencer à comprendre le principe du PHP. Il est exécuté par le serveur web (Apache, Nginx, etc.) pour générer dynamiquement des morceaux de contenu à l'intérieur du HTML.

Au moment d'une requête, on demande au serveur de nous renvoyer une page HTML. Si le serveur voit du PHP dans cette page HTML, alors il l'exécute avant de renvoyer le résultat final.

Le PHP n'est pas le seul langage capable de cette prouesse. En fait, tous les langages de programmation sont techniquement capables de générer du texte dans du HTML. Mais le PHP est le seul dont c'est historiquement le rôle principal depuis sa création ; tout son écosystème et toute sa culture sont donc essentiellement tournés vers cette fonction de génération de HTML. La plupart de vos programmes PHP, qu'ils soient constitués de dix ou cent mille lignes de code, mèneront au bout du compte à `echo` des choses au monde.

Projet - Robot Factory

Usine de robots

Qui n'a jamais rêvé de se fabriquer un robot pour l'aider dans ses tâches quotidiennes ?

Dans un fichier `index.php`, qui contiendra tout en haut votre logique PHP, puis le document HTML à proprement parler, vous allez développer un programme capable de créer un nouveau robot à chaque rafraîchissement de la page. Le robot saluera alors l'utilisateur et se présentera. Exemple :

« Salut, humain. Je suis VX-2345. »

Le nom du robot est généré aléatoirement au chargement de la page. Il se constituera toujours de deux lettres majuscules, d'un tiret et de quatre nombres.

Bonus 1

Après s'être présenté, le robot annoncera la date et l'heure actuelles au format suivant :

« Nous sommes le 30 07 2020, il est 11:59:30. »

Bonus 2

Après avoir annoncé la date et l'heure, le robot choisira un nombre aléatoire compris entre 1 et 10, et vous dira si c'est un nombre pair ou impair. Exemple :

« J'ai choisi le nombre 3. C'est un nombre impair. »

Bonus 3

Notre robot découvre le sens de l'humour. Il lui prendra alors l'idée de dire son nom à l'envers. Un robot du nom de VX-2345 dira alors :

« Mon nom à l'envers s'écrit 5432-XV. Ah. Ah. Ah. »

Bonus 4

Oups ! Il s'avère que notre robot a une chance sur trois de devenir un peu trop familier.

Deux fois sur trois, le robot dira « Voulez-vous une boisson chaude ? »

Une fois sur trois, le robot dira « Tu veux un cookie ? »

Bonus 5

Notre robot "poète" se découvre toujours plus de nouveaux talents. Il va se mettre à énoncer les cent premiers nombres de la suite de Fibonacci.

require

Diviser les fichiers

Si nous reprenons notre projet « Robot Factory », nous voyons un unique fichier `index.php` dans lequel on trouve du HTML, du PHP d'affichage, du PHP de logique...

En bref, notre fichier `index.php` mélange beaucoup de choses. Surtout, il mélange logique de coulisses et affichage HTML. On n'a pour l'instant qu'une page unique, et notre robot n'a pas encore beaucoup de fonctionnalités. Que se passera-t-il quand ce projet gagnera en complexité ?

Un seul fichier, ça ne va pas. En javascript on divisait notre code en de nombreux fichiers, et le PHP ne fait pas exception à cette règle.

require

Imaginons un fichier `date.php` :

```
<p>Nous sommes le <?= date('d m Y') ?></p>
```

Et ce fichier `index.php` :

```
<!DOCTYPE html>
<html lang="fr-FR">
  <head>
    <meta charset="UTF-8">
    <title>Hey</title>
  </head>
  <body>
    <h1>Salut !</h1>

    <?php require './date.php'; ?>
  </body>
</html>
```

Testez ce code, vous verrez qu'il fera ce à quoi vous pourriez vous attendre : le fichier `index.php` va inclure le contenu du fichier `date.php` précisément à l'endroit où on l'appelle via la fonction `require`. En disant les choses simplement, c'est comme un genre de copier-coller automatique que PHP fera au moment de son exécution. Le résultat sera exactement le même que si vous aviez dès le départ écrit ce code :

```
<!DOCTYPE html>
<html lang="fr-FR">
  <head>
    <meta charset="UTF-8">
    <title>Hey</title>
  </head>
  <body>
    <h1>Salut !</h1>

    <p>Nous sommes le <?= date('d m Y') ?></p>
  </body>
</html>
```

C'est ce que fait **require**.

Notez que comme pour **echo**, la fonction **require** peut s'appeler sans parenthèses. Ainsi **require './file.php'** fonctionne aussi bien que **require('./file.php')**.

Alternatives

La fonction **require** n'est pas la seule fonction PHP dont le rôle est d'inclure le contenu d'un fichier dans un autre fichier. Il y en a quatre au total :

- **require**
- **include**
- **require_once**
- **include_once**

La différence entre **require** et **include** se situe dans leur gestion des erreurs. En effet, si un bug apparaît dans un morceau de PHP intégré via **include**, un simple avertissement apparaîtra et le code poursuivra son exécution dans la mesure du possible. À l'inverse, le moindre souci dans un script intégré via **require** causera une erreur fatale et stoppera totalement l'exécution du programme.

La bonne pratique dans la plupart des cas est de préférer **require** à **include**. On ne souhaite pas poursuivre l'exécution du code en cas de bug, en PHP la philosophie est « ça marche ou ça marche pas », pas « ça marche à moitié ». Vous comprendrez bien vite pourquoi.

De leur côté, **require_once** et **include_once** fonctionnent comme leurs homonymes respectifs, à la différence qu'ils s'assurent que le fichier inclus n'a pas déjà été inclus une précédente fois. Là où **require** fonctionne comme un bête copier-coller et ne se pose aucune question, **require_once** n'inclut le fichier que s'il n'a pas déjà été inclus.

La fonction **require_once** a donc parfois son utilité, mais il faut savoir qu'elle est aussi moins performante que **require**. Dans la plupart des cas, il faut donc essayer de privilégier **require**, et éviter les inclusions multiples en amont grâce à un bon design et une bonne architecture.

index.php

Le fichier `index.php` est toujours le seul et unique point d'entrée de notre site web. On n'accède jamais directement aux autres fichiers. C'est pourquoi nos URL peuvent ressembler à `http://localhost/some_project/index.php`, ou même `http://localhost/some_project` car la plupart des serveurs web sont configurés pour chercher automatiquement un fichier `index.php` ou `index.html`.

Tout autre fichier que `index.php` sera uniquement appelé à travers `require`. Il ne sera jamais servi directement.

Dans ce cas, comment un site web peut-il avoir plusieurs pages ?

Grâce à un système de *routing*. Toujours à travers `index.php`, PHP sera capable de déterminer quels fichiers `require` en fonction de l'adresse demandée. Nous y viendrons bientôt.

Retenez, dans tous les cas, qu'on ne sert jamais directement un autre fichier que `index.php`. Des adresses du type `http://www.some-site.net/forum.php` existent sur le web, mais c'est une pratique ancienne et aujourd'hui mauvaise.

Les formulaires

Communiquer avec le serveur

Il y a plusieurs manières d'envoyer des requêtes HTTP à un serveur web. On en connaît bien sûr déjà une : entrer une adresse web dans le navigateur.

Lors de cette action, on envoie une requête de la méthode **GET** au serveur. On lui demande une ressource par son URI, il nous envoie une réponse.

Ici nous vous renvoyons au lointain cours que nous avons eu au sujet du fonctionnement du web. Il sera préférable pour vous de relire rapidement la partie sur les navigateurs et plus précisément les méthodes HTTP avant d'aller plus loin.

Blablabla méthode **GET**, donc, disions-nous. Quand on navigue sur le web, que ce soit en entrant directement des URI dans la barre d'adresse ou en cliquant sur des liens, pour l'essentiel on envoie des requêtes **GET**.

Il existe toutefois une action que vous connaissez très bien qui permet d'envoyer des requêtes de toutes les autres méthodes : soumettre un formulaire.

Détaillons cela.

Du formulaire HTML au traitement PHP

Vous l'avez peut-être déjà vu en intégration, vos formulaires HTML avaient d'étranges attributs que vous n'utilisiez peut-être pas encore : **action** et **method**. Les champs eux-mêmes avaient aussi un étrange attribut : **name**. Il est temps de vous révéler la vérité.

```
<form action="/handle-page" method="POST">
  <input type="text" name="nickname" placeholder="Entrez votre pseudonyme" />
  <input type="password" name="password" placeholder="Entrez votre mot de passe" />
  <button type="submit">Valider</button>
</form>
```

Quand on valide un formulaire, on envoie une requête au serveur. Une requête HTTP, c'est, pour ce qui nous intéresse ici : une adresse et des données.

L'adresse à laquelle le formulaire va envoyer la requête est spécifiée dans l'attribut `action`. Ici nous avons mis une adresse fictive `/handle-page`, qui est une URL relative indiquant « le chemin `/handle-page` à partir d'ici ».

L'attribut `action` n'est pas obligatoire. Si on l'omet, le formulaire envoie la requête à la même adresse que celle où il se trouve. C'est d'ailleurs ce qui est le plus souvent fait, et c'est ensuite côté PHP que l'on va déterminer si la requête demandait d'afficher le formulaire vide ou si celui-ci vient justement d'être soumis.

L'attribut `method` spécifie la méthode HTTP qui sera utilisée pour l'envoi de la requête au serveur. Nous allons détailler très vite les deux principales requêtes.

Ensuite, chaque champ va venir remplir la requête d'une donnée supplémentaire. Et l'attribut `name` est là pour donner un nom à chaque donnée. Vous pouvez imaginer qu'on envoie un tableau associatif de données au serveur, où chaque valeur entrée dans chaque champ sera associée à son `name` respectif.

Détaillons les méthodes pour mieux comprendre tout cela.

POST

Considérons à nouveau notre formulaire.

```
<form method="POST">
  <input type="text" name="nickname" placeholder="Entrez votre pseudonyme" />
  <input type="password" name="password" placeholder="Entrez votre mot de passe" />
  <button type="submit">Valider</button>
</form>
```

Oublions pour l'instant l'attribut `action`, il n'est pas important.

J'entre la valeur `Buffy` dans le champ `nickname`, et la valeur `Willow123` dans le champ `password`. Je sou mets le formulaire. Côté PHP, imaginons ce morceau de script sur cette même page

```
if (isset($_POST['nickname']) && isset($_POST['password'])) {
    echo $_POST['nickname'];
    echo '<br />';
    echo $_POST['password'];
}
```

La fonction `isset` permet simplement de savoir si une clé existe dans un tableau.

Ce qu'il faut noter ici, c'est l'usage de la variable `$_POST`. C'est ce qu'on appelle une « variable superglobale ». Il en existe un certain nombre. PHP les déclare pour nous : c'est-à-dire qu'elles existent et qu'on y a accès même si on ne les a pas déclarées nous-mêmes.

En l'occurrence, `$_POST` est toujours déclarée comme un tableau. Dans n'importe quel script PHP, vous pouvez appeler la variable `$_POST`, si aucun formulaire n'a été soumis elle contiendra un tableau vide.

Si un formulaire a été soumis avec la méthode **POST**, alors le tableau contenu dans la variable **\$_POST** ne sera plus vide. Nous aurons un tableau associatif rempli des valeurs entrées dans le formulaire.

Notre exemple plus haut affichera respectivement **Buffy** et **Willow123**. Notre tableau **\$_POST** contiendra exactement les mêmes valeurs qu'un tableau **\$values** dans le code suivant :

```
$values = [  
    'nickname' => 'Buffy',  
    'password' => 'Willow123'  
];
```

```
var_dump($values);
```

La fonction **echo** est très utile pour afficher des chaînes de caractères. À des fins de debug et de développement, elle peut être un peu limitante pour afficher des valeurs plus complexes, comme des tableaux ou des objets. Dans ce cas, n'hésitez pas à utiliser la fonction **var_dump**. Par exemple **var_dump(\$_POST)** dans notre cas. C'est un peu le **console.log** de PHP. La tester c'est l'adopter !

GET

La méthode GET fonctionne de la même manière, à deux différences près.

```
<form method="GET">  
    <input type="text" name="nickname" placeholder="Entrez votre pseudonyme" />  
    <input type="password" name="password" placeholder="Entrez votre mot de passe" />  
    <button type="submit">Valider</button>  
</form>  
if (isset($_GET['nickname']) && isset($_GET['password'])) {  
    echo $_GET['nickname'];  
    echo '<br />';  
    echo $_GET['password'];  
}
```

La première différence est évidente : les données ne sont pas accessibles via la variable superglobale **\$_POST**, mais via **\$_GET**.

La deuxième différence est par contre fondamentale. Tandis qu'une requête **POST** transporte les données de manière transparente, discrètement glissées dans ses métadonnées, une requête **GET** les transporte directement dans l'URI.

En soumettant le formulaire de notre exemple, notre URI ressemblera alors à ceci : **http://localhost/some_project?nickname=Buffy&password=Willow123**. Au lieu de transporter les données dans sa poche, la requête se les colle sur le front. (:

On trouve à la fin de notre adresse un premier séparateur **?**, qui dit « à partir d'ici on trouve les paramètres GET ». Puis chaque paire **clé=valeur** est séparée par un **&**. Le serveur est ensuite capable de les lire et les utiliser. Dans le cas des PHP, on les trouve dans la variable **\$_GET** sous forme de tableau associatif.

Il y a des raisons à cela. Comparons les deux méthodes.

Quand utiliser GET ? Quand utiliser POST ?

Philosophiquement, une adresse web est là pour décrire ou demander une ressource. La méthode GET est généralement utilisée pour demander à voir quelque chose, c'est d'ailleurs pour cela que cliquer sur un lien ou aller directement à une adresse déclenche une requête GET. Mais on l'utilise aussi, par exemple, pour les formulaires de recherche. Si vous soumettez la recherche **Tiramisu** sur Qwant, les résultats s'afficheront et l'adresse ressemblera à ceci : <https://www.qwant.fr?q=Tiramisu>. On peut deviner ici que **q** signifie *query*. Le serveur de Qwant aura alors utilisé la valeur de **q** pour faire tourner ses algorithmes de recherche sur sa base de données, et nous envoyer une réponse appropriée.

Si **GET** sert à demander au serveur de nous envoyer quelque chose, **POST** est plutôt là pour envoyer quelque chose au serveur. Un formulaire d'inscription, par exemple, utilisera la méthode **POST**.

Bien sûr, en soi, on peut très bien intervertir les deux. On peut créer des utilisateurs avec un formulaire GET, on peut faire des recherches avec un formulaire POST. C'est avant tout une question de sémantique. Et la sémantique, vous commencez à le savoir, c'est important.

Il existe d'autres méthodes, comme **PATCH** ou **DELETE**. En théorie il faudrait les utiliser. **DELETE**, par exemple, sert pour un formulaire visant à supprimer une ressource, comme un utilisateur ou une publication. Mais en pratique, la multiplicité des requêtes complexifie beaucoup le développement, aussi la plupart des développeurs se limitent à **GET** et **POST** par souci de simplicité. Même Symfony, l'un des *frameworks* PHP les plus utilisés au monde et objectivement l'un des mieux pensés, ne se fait pas de nœuds au cerveau concernant les méthodes HTTP : **GET** et **POST**.

Vous concernant, même si vous estimez être un chevalier de la sémantique et rêvez d'utiliser un jour exactement la bonne méthode pour la bonne requête, nous vous recommandons pour l'heure de vous limiter également à **GET** et **POST**. En vérité, dans votre vie professionnelle, vous ne serez amenés à véritablement respecter la sémantique exacte des méthodes HTTP que lorsque vous développerez des API REST, chose que vous ne ferez probablement pas à vos débuts.

Projet - Robot Factory 2

Robots sur mesure

Notre usine de robots est très fonctionnelle. Elle présente toutefois un problème majeur : nous n'avons aucun contrôle sur la construction des robots. Leurs noms sont générés aléatoirement, par exemple.

Quand l'utilisateur arrive sur la page, il verra désormais un formulaire de réglage de robot. Ce formulaire demandera à l'utilisateur de saisir un nom pour son robot. Puis lorsque l'utilisateur valide le formulaire, la page se recharge et le robot est créé avec le nom ainsi entré.

Notez que ce champ n'est pas requis. Si l'utilisateur valide le formulaire sans saisir de nom, alors celui-ci sera toujours généré aléatoirement.

Pour le reste, le robot effectue toujours les actions du précédent cours : dire son nom, choisir un nombre, dire son nom à l'envers, etc.

Oh ! Et bien sûr, cette fois on divise les fichiers ! Nous pourrions imaginer le découpage suivant :

- Un fichier `robot_functions.php` dédié aux fonctions propres au robot
- Un fichier `generic_functions.php` dédié aux fonctions génériques
- Un fichier `homepage.phtml` dédié au template HTML à proprement parler. L'extension `phtml` sera utilisée par convention pour les fichiers contenant essentiellement du HTML mais aussi un peu de PHP.
- Un fichier `index.php`, point d'entrée du programme, servant de jonction entre tous les autres fichiers

Simple ! C'est parti.

Indices

- Vous aurez besoin de rendre votre HTML dynamique : vous affichez le formulaire si et seulement s'il n'a pas déjà été soumis, sinon vous affichez le robot nouvellement créé. Jetez un œil à la **syntaxe alternative des structures de contrôle**
- N'oubliez pas la fonction `isset`. Vous pourriez aussi avoir besoin de la fonction `empty`.

Bonus 1

Mon robot est-il familier ? L'utilisateur a désormais la possibilité de ne plus se fier au destin pour ce choix.

Un champ de formulaire permet à l'utilisateur de spécifier la familiarité de notre robot. S'il le souhaite, il a toujours la possibilité de s'en remettre au hasard.

Bonus 2

Si notre robot est familier, la dernière phrase s'affiche en rouge et en gras.

Bonus 3

Si l'utilisateur saisit un nom de robot, obligez-le à respecter le format « XX-9999 », c'est-à-dire deux lettres, un tiret et quatre chiffres. Si le nom saisi ne correspond pas au format demandé, empêchez la création du robot.

Ce bonus est d'un niveau un peu avancé, même pour les bons élèves. Ce n'est pas grave si vous préférez plutôt améliorer librement votre robot, ou aider vos camarades. Si vous tenez malgré tout à accomplir ce bonus 3, n'hésitez pas à demander de l'aide à vos formateurs.

Bonus 4

Pour nous assurer que notre robot ne tente pas de détruire notre civilisation, il va falloir l'occuper.

Vous allez jouer au juste prix avec lui. Dans votre tête, vous choisirez un nombre compris entre 1 et 1000. Le robot aura pour mission de deviner ce nombre. À chaque nouvelle tentative de sa part, vous lui direz « C'est plus » ou « C'est moins » via un formulaire.

Robot Factory

Ma proposition : 149

C'est plus

C'est moins

C'est juste

Quand il aura trouvé le nombre, vous pourrez également le lui signifier et il exprimera alors sa joie immense.

Il va de soi qu'il doit essayer de deviner votre nombre en un minimum de tentatives.

Indice : recherche dichotomique. (: